

Trabajo Práctico N.º 3

Estimación y reducción de potencia en un interpolador FIR

Inserción de ceros frente a descomposición polifásica

Curso de Diseño Digital VLSI – Fundación Fulgor

Damián Lugano

25 de octubre de 2026

Índice

1. Introducción **1**
1.1. Objetivos . . . 1
1.2. Ejercicios solicitados . . . 1

2. Marco teórico **1**
2.1. Interpolación mediante inserción de ceros . . . 1
2.2. Descomposición polifásica . . . 2
2.3. Componentes de la potencia . . . 2
2.4. Glitches y actividad real . . . 2

3. Ejercicio 1: implementación, verificación y cierre de timing **2**
3.1. Parámetros comunes . . . 3
3.2. Modelo de referencia en Python . . . 3
3.3. Implementaciones RTL . . . 6
3.4. Conteo de multiplicaciones . . . 8
3.5. Verificación RTL . . . 8
3.6. Barrido de síntesis y frecuencia máxima . . . 9
3.7. Frecuencia y período de trabajo comunes . . . 10
3.8. Simulación a nivel de compuertas (GLS) . . . 10
3.9. Estimación de potencia con actividad de GLS . . . 10

1. Introducción

1.1. Objetivos

El objetivo de este trabajo es implementar dos arquitecturas RTL de un mismo interpolador FIR y comparar su consumo de potencia. La primera arquitectura eleva la tasa de muestreo mediante la inserción explícita de ceros y procesa la secuencia resultante con un FIR convencional. La segunda utiliza una descomposición polifásica para evitar las multiplicaciones cuyos operandos se conocen nulos.

Ambas implementaciones deben producir exactamente la misma secuencia de salida. Esta equivalencia funcional es una condición necesaria para que las diferencias posteriores de potencia, área y timing puedan atribuirse a la arquitectura y no a una diferencia en el procesamiento numérico.

Además de medir el consumo total, se busca comprender qué estructuras lo originan, separar las componentes dinámica y estática, observar el efecto de los glitches y evaluar técnicas de reducción de potencia sin perder equivalencia funcional ni incumplir las restricciones de timing.

1.2. Ejercicios solicitados

El trabajo se organiza en los siguientes ejercicios:

1. **Implementación, verificación y cierre de timing.** Desarrollar el modelo fixed-point, implementar ambas arquitecturas, demostrar su equivalencia y determinar un período de trabajo común.
2. **Anatomía del consumo.** Separar potencia de switching, potencia interna y leakage, e identificar los bloques que dominan el consumo.
3. **Glitches.** Comparar la actividad espuria de las arquitecturas mediante simulaciones de compuertas con y sin retardos anotados.
4. **Data gating.** Reducir la actividad de la arquitectura directa evitando la conmutación de operaciones cuyo resultado se conoce de antemano.
5. **Clock gating.** Detener selectivamente el clock de registros que no necesitan actualizarse y cuantificar el ahorro obtenido.
6. **Multi-V_t.** Evaluar el compromiso entre timing y leakage al utilizar celdas con distintas tensiones de umbral.
7. **Arquitecturas opcionales.** Analizar el compromiso entre hardware compartido, paralelismo y aprovechamiento de la simetría de los coeficientes.

2. Marco teórico

2.1. Interpolación mediante inserción de ceros

Para elevar la tasa de muestreo por un factor entero L, se construye una secuencia $x_u[n]$ que intercala $L-1$ ceros entre muestras consecutivas:

$$x_u[n] = \begin{cases} x[n/L], & n \equiv 0 \pmod{L}, \\ 0, & \text{en otro caso.} \end{cases} \tag{1}$$

La secuencia interpolada se obtiene filtrando $x_u[n]$ con el filtro prototipo de N taps:

$$y[n] = \sum_{k=0}^{N-1} h[k] x_u[n-k]. \tag{2}$$

El FIR directo trabaja a la tasa alta Lf_s y evalúa sus N productos para cada muestra de salida. Sin embargo, debido a los ceros insertados, solamente N/L de los datos almacenados son distintos de cero en cada fase.

2.2. Descomposición polifásica

Escribiendo el índice de salida como $n = mL + p$, donde p identifica la fase, la ecuación anterior puede reorganizarse como

$$y[mL + p] = \sum_{r=0}^{N/L-1} h[rL + p]x[m - r]. \tag{3}$$

Se define entonces un subfiltro por fase,

$$E_p[r] = h[rL + p], \qquad p = 0, \ldots, L - 1. \tag{4}$$

Cada subfiltro posee $N/L = 4$ taps y procesa únicamente las muestras originales a la tasa f_s. Las cuatro respuestas se calculan en paralelo y un selector entrega una fase por ciclo del clock de salida.

La cantidad de multiplicaciones por segundo de ambas organizaciones es

$$R_{\text{directa}} = NLf_s, \qquad R_{\text{poli}} = \frac{N}{L} Lf_s = Nf_s. \tag{5}$$

Por lo tanto, el ahorro aritmético teórico de la forma polifásica es un factor L. Este factor no se traduce necesariamente de forma directa a potencia total, ya que también intervienen la red de clock, los registros, la lógica de control y el área.

2.3. Componentes de la potencia

La potencia total de un circuito CMOS se separa en

$$P_{\text{total}} = P_{\text{switching}} + P_{\text{internal}} + P_{\text{leakage}}. \tag{6}$$

La potencia de switching corresponde a la energía necesaria para cargar y descargar las capacidades de las interconexiones y entradas de las celdas. Una aproximación habitual es

$$P_{\text{switching}} = \alpha C_L V_{DD}^2 f_{\text{clk}}, \tag{7}$$

donde α representa la actividad de conmutación. La potencia interna se disipa dentro de las celdas debido a la carga de nodos internos y a la conducción simultánea de transistores durante las transiciones. Ambas forman la potencia dinámica y dependen de la actividad, la frecuencia, los slews y las cargas.

La potencia de leakage está presente incluso cuando el circuito no conmuta. Depende del tipo de celda, su estado lógico, la tensión de alimentación y la temperatura, y crece con el área implementada. Para comparar diseños que realizan distinto trabajo por ciclo se utiliza también la energía por muestra de salida:

$$E_{\text{muestra}} = \frac{P_{\text{total}}}{Lf_s}. \tag{8}$$

2.4. Glitches y actividad real

Los glitches son transiciones espurias producidas cuando señales que convergen en una compuerta llegan con retardos diferentes. Son frecuentes en multiplicadores y árboles de sumadores, donde existen numerosos caminos combinacionales de distinta profundidad. Para que un glitch aparezca deben existir convergencia de señales, desbalance de tiempos de llegada, sensibilización lógica y un pulso suficientemente largo para superar el filtrado inercial.

Una simulación RTL sin retardos no permite observar este fenómeno. Por ese motivo, las estimaciones de potencia principales se realizarán con actividad obtenida de una simulación de compuertas con SDF. El estímulo utilizado también resulta determinante: comparar arquitecturas exige aplicar la misma secuencia y el mismo período de clock.

3. Ejercicio 1: implementación, verificación y cierre de timing

El objetivo del primer ejercicio es implementar las dos arquitecturas, demostrar que son funcionalmente equivalentes y preparar el flujo para determinar el período mínimo, realizar GLS y obtener la primera estimación de potencia.

3.1. Parámetros comunes

La Tabla 1 resume los parámetros empleados por las dos arquitecturas.

Cuadro 1: Parámetros comunes del interpolador.

Parámetro	**Valor**
Factor de interpolación, L	4
Cantidad de taps, N	16
Formato de entrada	signed Q1.11, 12 bits
Formato de coeficientes	signed Q1.11, 12 bits
Formato de salida	signed Q1.11, 12 bits
Cantidad de muestras de entrada	2000
Cantidad de muestras de salida	8015
Tasa del clock	Lf_s

3.2. Modelo de referencia en Python

Diseño y cuantización del filtro

El filtro prototipo se diseñó mediante `firwin`, utilizando una ventana de Hamming y una frecuencia de corte normalizada igual a $1/L$. Su ganancia se multiplicó por L para compensar la reducción de amplitud introducida por la inserción de ceros. Los coeficientes se redondearon al formato signed Q1.11.

Los coeficientes enteros resultantes son

$$h = [-11, -44, -101, -88, 167, 740, 1462, 1971, \\ 1971, 1462, 740, 167, -88, -101, -44, -11]. \tag{9}$$

La distribución de estos coeficientes entre las cuatro fases se muestra en la Tabla 2.

Cuadro 2: Coeficientes enteros Q1.11 de los subfiltros polifásicos.

Fase	$E_p[0]$	$E_p[1]$	$E_p[2]$	$E_p[3]$
0	-11	167	1971	-88
1	-44	740	1462	-101
2	-101	1462	740	-44
3	-88	1971	167	-11

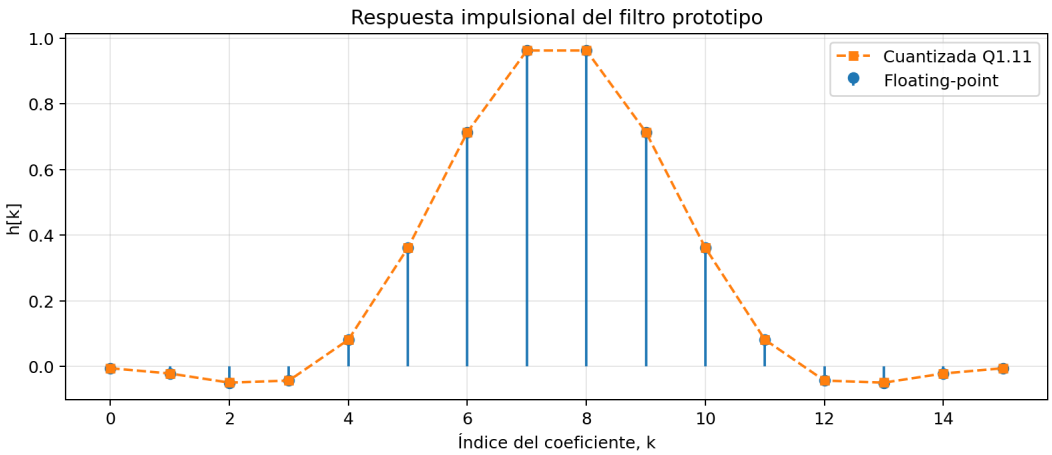


Figura 1: Respuesta impulsional del filtro prototipo antes y después de la cuantización.

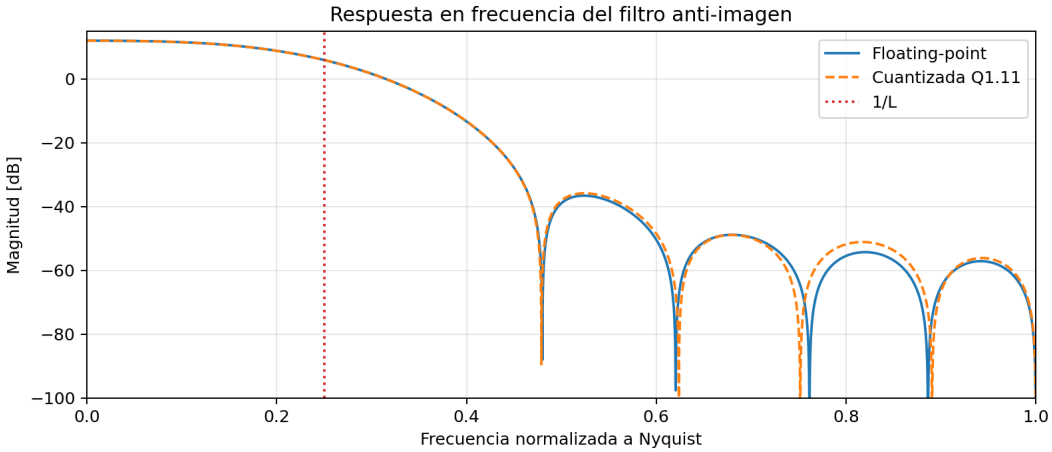


Figura 2: Respuesta en frecuencia del filtro pasabajos anti-imagen.

Aritmética de punto fijo

Las muestras y los coeficientes se representan con 12 bits y 11 bits fraccionarios. Cada multiplicación genera un producto signed de 24 bits con 22 bits fraccionarios, que luego serán los operandos del arbol de sumas, al cual se dejará desarrollar su resolución completa.

Cuadro 3: Formatos internos utilizados por el RTL.

Magnitud	**Bits totales**	**Bits fraccionarios**
Entrada y coeficiente	12	11
Producto	24	22
Acumulador directo, 16 taps	28	22
Acumulador por fase, 4 taps	26	22
Salida	12	11

La conversión a la salida trunca los bits fraccionarios menos significativos y satura cuando la magnitud excede el rango signed de 12 bits.

Estímulo de entrada común

El estímulo está compuesto por dos tonos, ruido blanco de baja amplitud y una normalización del pico a $-6\,\mathrm{dBFS}$:

$$x[n] = \sin(2\pi\,0.073n) + 0.63\sin(2\pi\,0.181n + 0.37) + 0.04w[n]. \tag{10}$$

Se emplea una semilla fija para el rng y luego se cuantiza la señal a Q1.11. De esta manera el estímulo es reproducible y puede reutilizarse sin modificaciones en las simulaciones RTL, GLS y en todas las estimaciones de potencia.

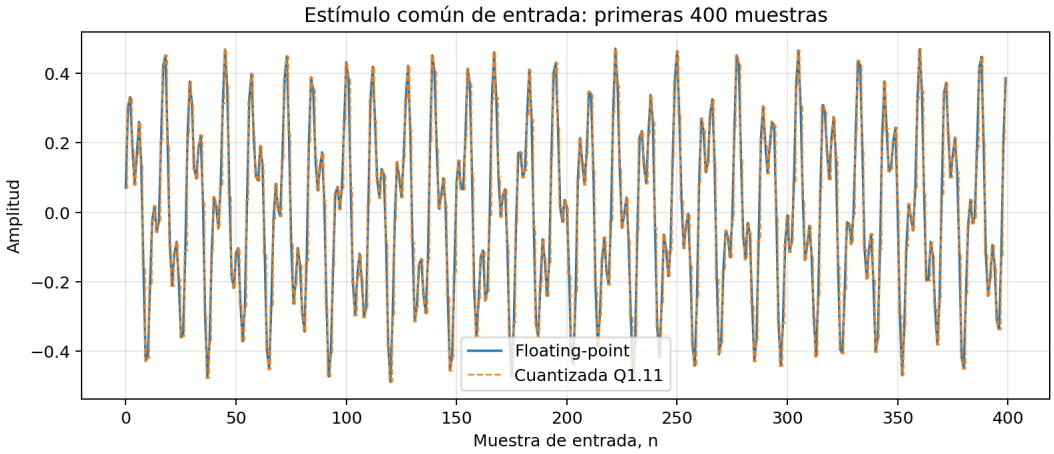


Figura 3: Señal de entrada utilizada como estímulo común.

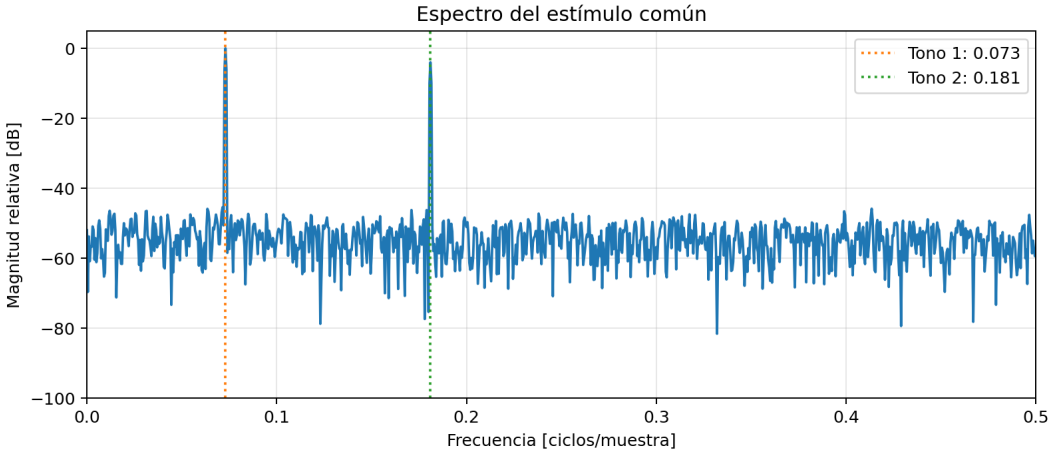


Figura 4: Espectro del estímulo común.

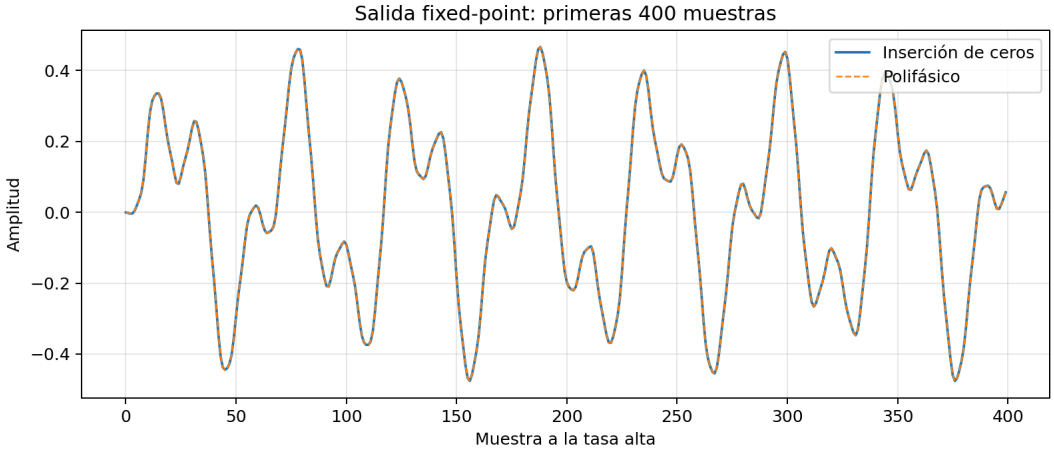


Figura 5: Salida fixed-point del interpolador.

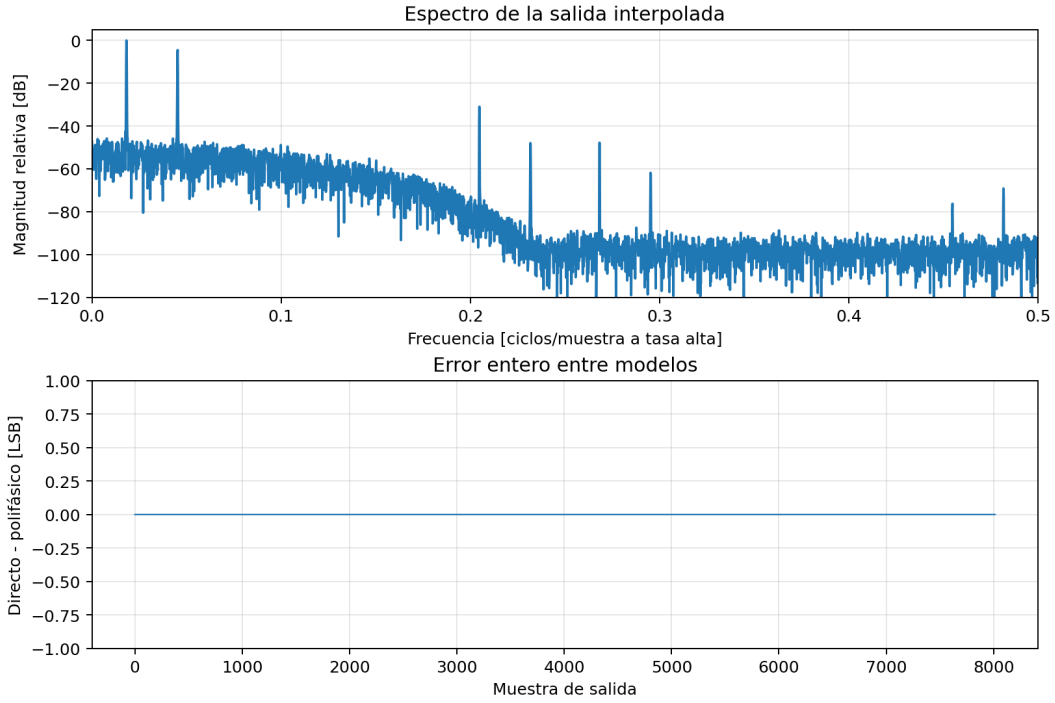


Figura 6: Espectro de salida y error muestra a muestra entre los dos modelos.

Equivalencia de los modelos

El modelo implementa por separado la interpolación mediante inserción de ceros y la forma polifásica de la Ecuación 3. Ambas rutas utilizan los mismos coeficientes cuantizados y la misma aritmética de punto fijo. La comparación muestra a muestra confirmó que las dos generan exactamente la misma secuencia, sin diferencias de cuantización.

Para 2000 muestras de entrada, el archivo de referencia contiene 8015 muestras de salida. Esta longitud incluye la respuesta transitoria final de la convolución y los ceros necesarios para conservar la misma convención en las dos formas del modelo.

3.3. Implementaciones RTL

Interfaz común

Las dos arquitecturas exponen la misma interfaz. El único clock trabaja a la tasa Lf_s y `i_valid` indica la llegada de una muestra nueva a f_s, por lo que permanece alto un ciclo de cada cuatro. Después de completar la latencia inicial, `o_valid` acompaña una muestra de salida en todos los ciclos.

La interfaz RTL común se declara de la siguiente forma. En `interp_poly` solamente cambian el nombre del módulo y el paquete propio importado; la lista de puertos es idéntica.

Listing 1: Interfaz RTL común de los interpoladores.

```
module interp
    import interp_pkg::*;
    import fir_pkg::*;
(
    input  logic  i_clk,    // Clock a L*f_s
    input  logic  i_rst,    // Reset asincronico activo en alto
    input  logic  i_valid,  // Muestra nueva, un ciclo de cada L
    input  data_t i_data,   // Muestra signed Q1.11

    output logic  o_valid,  // Indica una salida valida
    output data_t o_data    // Resultado signed Q1.11
);
```

FIR compartido

Ambas implementaciones reutilizan el mismo módulo `fir.sv` ya que, a fin de cuentas, se trata conceptualmente del mismo filtro. Sin embargo, cada variante posee un paquete propio que determina la cantidad de taps y, por lo tanto, los tipos de los buses internos y el ancho del acumulador. En el modulo FIR se encuentra el shift register, los productos parciales, la suma con precisión completa y el bloque común de truncamiento y saturación.

En la implementación directa el paquete configura 16 taps. En la polifásica se utilizan cuatro instancias del mismo RTL, cada una configurada con 4 taps y con los coeficientes de una fase diferente.



Figura 7: Arquitectura RTL del filtro FIR genérico.

Interpolador mediante inserción de ceros

La arquitectura `interp_zeros` sustituye `i_data` por cero durante los ciclos en los que `i_valid` está inactivo. El FIR permanece habilitado en todos los ciclos y, por lo tanto, su línea de retardo contiene explícitamente los tres ceros insertados entre muestras consecutivas.

La primera entrada válida se incorpora a la línea de retardo en el ciclo en el que se recibe. El registro de salida del FIR captura el primer resultado en el ciclo siguiente, por lo que la latencia de la arquitectura es de un ciclo del clock de alta tasa. A partir de ese momento se produce una salida válida por ciclo.

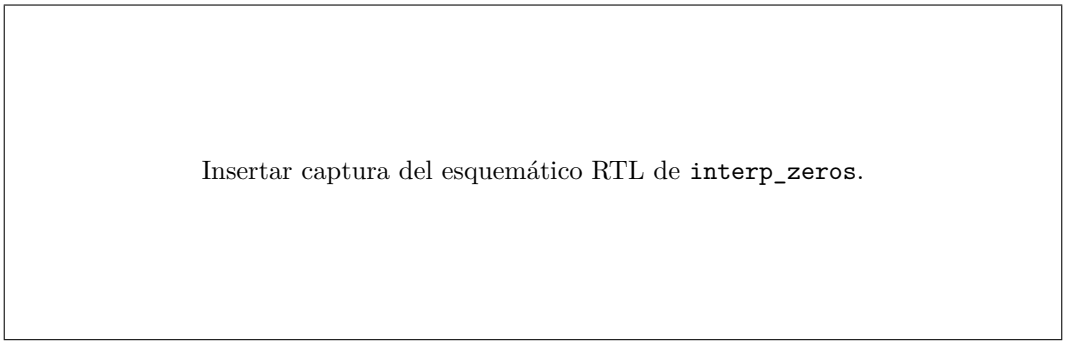


Figura 8: Arquitectura RTL del interpolador mediante inserción de ceros.

Interpolador polifásico

La arquitectura `interp_poly` instancia cuatro FIR de cuatro taps. Todos reciben la misma muestra de entrada y avanzan simultáneamente cuando `i_valid` está activo. Como este pulso aparece una vez cada cuatro ciclos, cada subfiltro trabaja efectivamente a la tasa f_s. Sus salidas registradas se entregan mediante un selector que recorre las fases 0, 1, 2 y 3.

El FIR actualiza el resultado correspondiente a una muestra en el siguiente evento de habilitación. Como estos eventos están separados por cuatro clocks, la primera salida de la arquitectura polifásica aparece cuatro ciclos después de aceptar la primera entrada. Luego el selector entrega una salida por ciclo sin interrupciones.

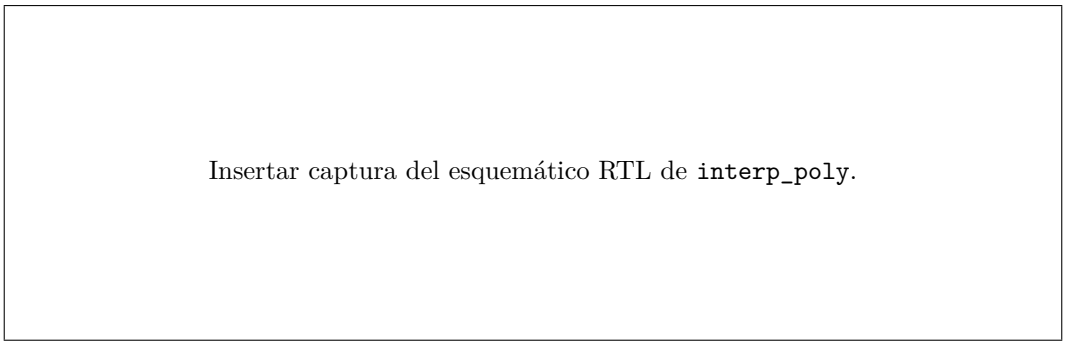


Figura 9: Arquitectura RTL del interpolador polifásico.

Latencia y alineación

La Tabla 4 resume las latencias observadas desde el primer `i_valid`. La diferencia de tres ciclos no modifica la secuencia numérica y se resuelve en la verificación utilizando `o_valid`, sin introducir una compensación específica para cada DUT.

Cuadro 4: Latencia de las implementaciones RTL actuales.

Arquitectura	**Primera salida válida**	**Latencia**
`interp_zeros`	Ciclo 1	1 clock de Lf_s
`interp_poly`	Ciclo 4	4 clocks de Lf_s

3.4. Conteo de multiplicaciones

Ambas descripciones RTL contienen 16 multiplicadores, uno por tap en el FIR directo y cuatro por cada una de las cuatro fases en el polifásico. Sin embargo, la diferencia está principalmente en la tasa de productos, el FIR directo hace 16 productos por cada salida a Lf_s, aunque solamente cuatro operandos de datos son distintos de cero. La arquitectura polifásica hace 16 productos en paralelo una vez cada 4 ciclos, equivalentes a 4 productos por cada salida a Lf_s.

Cuadro 5: Conteo estructural y actividad aritmética.

Arquitectura	**Multiplicadores RTL**	**Productos por salida**	**Tasa de productos**
Inserción de ceros	16	16	$16Lf_s = 64f_s$
Polifásica	16	4	$4Lf_s = 16f_s$

Por lo tanto, la arquitectura polifásica reduce en un factor $L = 4$ la cantidad de operaciones por unidad de tiempo. Este es un conteo de la descripción RTL; la implementación final de los productos con coeficientes constantes se verificará en los reportes de síntesis.

3.5. Verificación RTL

Entorno de verificación común

Se adaptó un entorno basado en clases compuesto por generador, driver, monitor y scoreboard. El mismo testbench se utiliza para ambas arquitecturas; solamente cambia el módulo seleccionado durante la compilación.

El generador lee las 2000 entradas cuantizadas exportadas por el modelo de Python. Cada muestra se aplica con `i_valid` activo durante un ciclo y se insertan tres ciclos inactivos antes de la muestra siguiente. Al finalizar se envían muestras nulas adicionales, manteniendo el mismo protocolo, para vaciar las líneas de retardo.

El monitor ignora la latencia particular del DUT y publica una muestra únicamente cuando `o_valid` está activo. El scoreboard lee las 8015 palabras del archivo golden y las compara, informando frente a una eventual diferencia.

Resultados

La simulación RTL de ambas implementaciones finalizó correctamente. Para cada DUT, el scoreboard comparó las 8015 muestras válidas contra el mismo archivo de referencia y no detectó diferencias. De esta forma se confirmó la equivalencia bit a bit entre `interp_zeros`, `interp_poly` y el modelo fixed-point de Python.

La comparación comienza con la primera activación de `o_valid`; por lo tanto, la diferencia de tres ciclos entre las latencias de los DUT no requiere modificar el estímulo ni mantener dos secuencias esperadas.

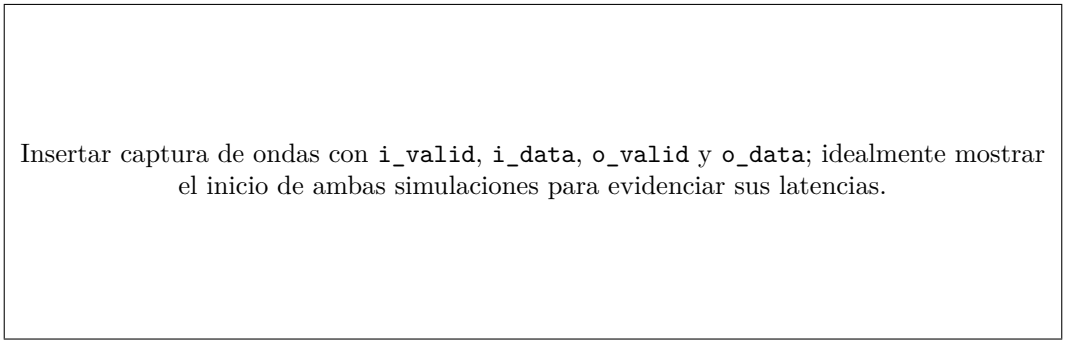


Figura 10: Inicio de la simulación RTL y primera salida válida de cada arquitectura.

3.6. Barrido de síntesis y frecuencia máxima

Se sintetizó cada arquitectura con períodos de clock decrecientes a partir de 1ns. La Tabla 6 resume los reportes del escenario `func_slow`. Se presentan el WNS y el TNS, junto con la cantidad de caminos en violación de setup.

Cuadro 6: Barrido de síntesis.

Arquitectura	f [GHz]	T [ns]	WNS [ns]	TNS [ns]	n_{path}
Inserción de ceros	1	1.000	0.00	0.00	0
	2	0.500	0.00	0.00	0
	2.5	0.400	−0.00	−0.00	1
	3	0.333	−0.00	−0.00	1
	4	0.250	−0.00	−0.00	4
Polifásica	1	1.000	0.00	0.00	0
	2	0.500	0.00	0.00	0
	3	0.333	0.00	0.00	0
	3.5	0.285	−0.00	−0.00	3
	4	0.250	−0.00	−0.00	7

Según los resultados obtenidos en los reportes del barrido de sintesis, las frecuencias máximas sin violaciones de timing para cada arquitectura son:

$$f_{\text{máx,zeros}} \approx 2.5\,\text{GHz}, \qquad f_{\text{máx,poly}} \approx 3\,\text{GHz}. \tag{11}$$

Caminos críticos

En `interp_zeros`, el recorrido crítico predominante parte de un registro de la línea de retardos del FIR, atraviesa la lógica aritmética de los productos por coeficientes constantes, el arbol de sumas y termina en el registro de salida del FIR. En el reporte, el punto de partida exacto es `u_fir/data_r_reg` y el de llegada `u_fir/o_data_reg`.

En `interp_poly`, el camino crítico es el que conecta los registros de muestras con los registros de resultado de las fases. En el reporte, parte de `gen_phases.u_fir_phase/data_r_reg` y termina en `gen_phases.u_fir_phase/o_data_reg`. La división en subfiltros de cuatro taps reduce la profundidad del arbol de sumas respecto del FIR directo de 16 taps. Esto permite alcanzar una frecuencia máxima mayor en el barrido realizado.

3.7. Frecuencia y período de trabajo comunes

Se fija como frecuencia de trabajo en común la $f_{\text{clk}} = 2\,\text{GHz}$, ya que deja un margen de aproximadamente 20 % respecto a la máxima frecuencia para la cual la arquitectura directa cierra timing. Este margen se deja para conservar espacio para las modificaciones posteriores y, al mismo tiempo, evitar una síntesis casi exclusivamente HVT como la observada a 1 GHz.

La Tabla 7 muestra que a 2 GHz ambas síntesis utilizan los cuatro tipos de celdas. Los porcentajes corresponden a cantidad de celdas, según `vt_groups.rpt`.

Cuadro 7: Distribución de celdas por tensión de umbral a la frecuencia común de 2 GHz.

Arquitectura	HVT [%]	RVT [%]	LVT [%]	SLVT [%]
Inserción de ceros	69.80	1.41	20.08	8.71
Polifásica	73.60	0.79	20.22	5.40

La presencia de celdas LVT y SLVT deja una oportunidad para estudiar el compromiso entre leakage y timing mediante multi-V_t en el Ejercicio 6.

3.8. Simulación a nivel de compuertas (GLS)

En esta etapa se ejecutó la simulación a nivel de compuertas sobre ambas arquitecturas con los retardos SDF anotados. En un primer momento, al ejecutar `make gls`, aparecieron las violaciones de ancho de pulso `$width` advertidas en la consigna, asociadas a los límites genéricos de los modelos de librería. Por ejemplo, en registros del FIR directo se informó un límite de 1000 ps frente a intervalos de 250 ps. A la frecuencia de trabajo de 2 GHz, el período es de 500 ps y el semiperíodo nominal es de 250 ps, por lo que ese límite de 1 ns se incumple aun con el clock previsto para el diseño.

Estos mensajes se identificaron como falsas violaciones debidas al placeholder de ancho de pulso del modelo, por lo que no corresponden a violaciones de setup o hold del camino de datos.

Luego se ejecutó `make gls-ntc`, que deshabilita los chequeos de timing mediante `+notimingcheck` y conserva los retardos SDF. Ambas arquitecturas completaron correctamente la simulación y la comparación bit a bit con el modelo de referencia. Estas corridas se utilizaron para obtener la actividad de potencia.

3.9. Estimación de potencia con actividad de GLS

En esta etapa, se convirtió a SAIF la actividad obtenida de GLS con SDF y se ejecutó PrimeTime PX en modo de potencia promedio. La comparación utiliza el mismo estímulo y una frecuencia común de 2 GHz. El reporte de actividad confirmó una cobertura de anotación del 100 % en ambas arquitecturas.

Cuadro 8: Potencia estimada con PrimeTime PX a 2 GHz y actividad de GLS con SDF.

Magnitud	**Inserción de ceros**	**Polifásica**	**Reducción [%]**
Switching [mW]	4.516	2.351	47.9
Internal [mW]	1.964	0.5207	73.5
Leakage [μW]	17.68	14.24	19.5
Potencia total [mW]	6.497	2.886	55.6
Energía por salida [pJ]	3.249	1.443	55.6

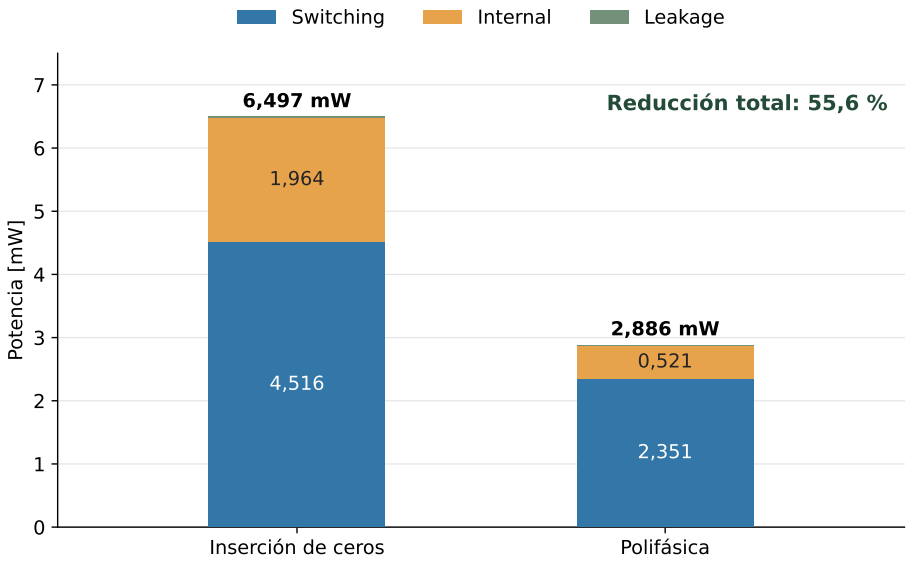


Figura 11: Comparación de potencia por componente a 2 GHz con actividad de GLS y retardos SDF.

La energía por muestra se calculó dividiendo la potencia total por la tasa de salida, que coincide con la frecuencia de clock de 2 GHz. La arquitectura polifásica reduce la potencia total un 55.6 %, con una relación

$$\frac{P_{\text{zeros}}}{P_{\text{poly}}} = \frac{6.497}{2.886} \approx 2.25. \tag{12}$$

Este factor es menor que el ahorro aritmético teórico $L = 4$, ya que la potencia total también incluye el consumo de registros y de la distribución de clock. Además, la arquitectura polifásica incorpora lógica de selección y control de fases, cuyos consumos no escalan proporcionalmente con la cantidad de productos por muestra. La reducción relativa más marcada se observa en la potencia interna, del 73.5 %, mientras que la potencia de switching disminuye un 47.9 %. El leakage se reduce un 19.5 % y representa menos del 0.5 % de la potencia total en ambas variantes; por lo tanto, en estas condiciones domina la potencia dinámica.

Cuadro 9: Desglose de potencia total por grupo a 2 GHz.

Grupo	**Zeros [mW]**	**Poly [mW]**	**Cambio [%]**
Combinacional	4.993	1.576	−68.4
Registros	0.8891	0.5678	−36.1
Red de clock	0.6148	0.7418	+20.7

En base al desglose de potencia por grupos, se puede observar que la lógica combinacional concentra el 76.85 % del consumo de *zeros* y el 54.61 % del de *poly*. Su reducción de aproximadamente 3.417 mW explica la mayor parte del ahorro absoluto. Esto evidencia que la arquitectura polifásica genera un ahorro considerable al calcular únicamente los productos que efectivamente contribuyen a cada muestra de salida.

Los registros también consumen menos en la arquitectura polifásica, aunque su reducción es menor. En cambio, la red de clock aumenta un 20.7 % en potencia absoluta y pasa de representar el 9.46 % al 25.71 % del total. Este incremento se debe a que la reorganización del procesamiento en fases modifica la estructura secuencial del circuito y aumenta la carga efectiva asociada al clock, aun cuando la lógica combinacional total se reduzca.